

Step 1: Defining the Playground

Functional Requirements (The 'What')

- [] Create posts (Text/Media)
- [] Follow/Unfollow users
(Unidirectional)
- [] View feed in chronological
order
- [] Pagination (Infinite Scroll)

Non-Functional Requirements (The 'How')

Scale: 2 Billion Users

Latency: < 500ms (P99 for
Posting & Viewing)

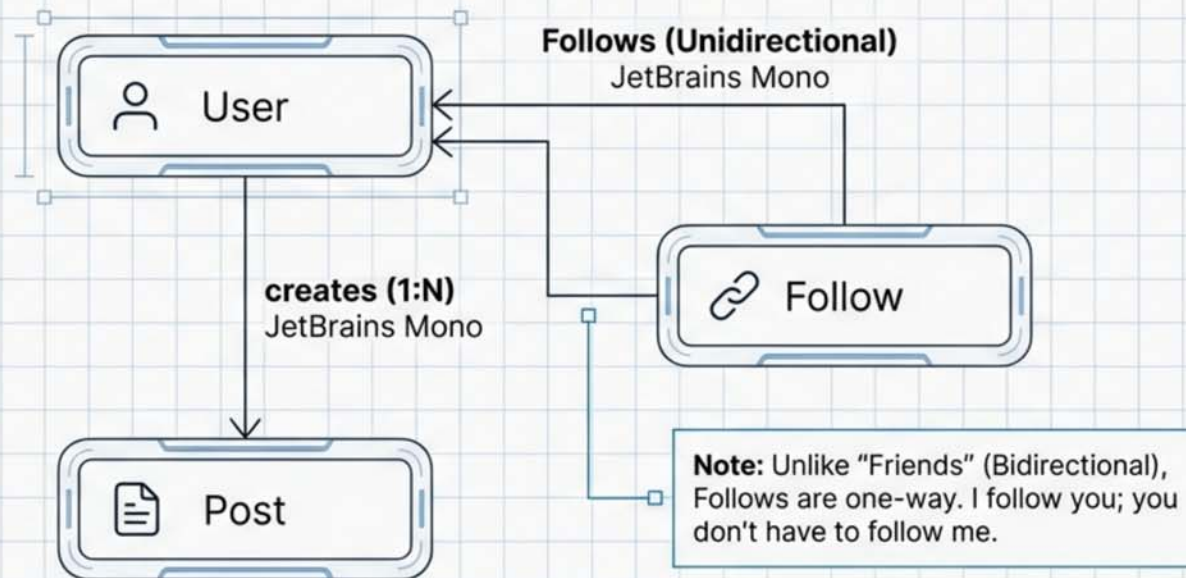
Consistency model: Eventual
Consistency

Staleness Allowance: < 1 Minute

Key Insight: Quantifying 'staleness' (e.g., 1 minute) is the contract that allows us to use asynchronous processing later.

Step 2: Core Entities

The Nouns of the System



Step 3: API Design

```
// 1. Create Post
POST /posts
Response: 201 OK { postId: string }

// 2. Follow User (Idempotent)
PUT /users/{userId}/followers
Response: 200 OK

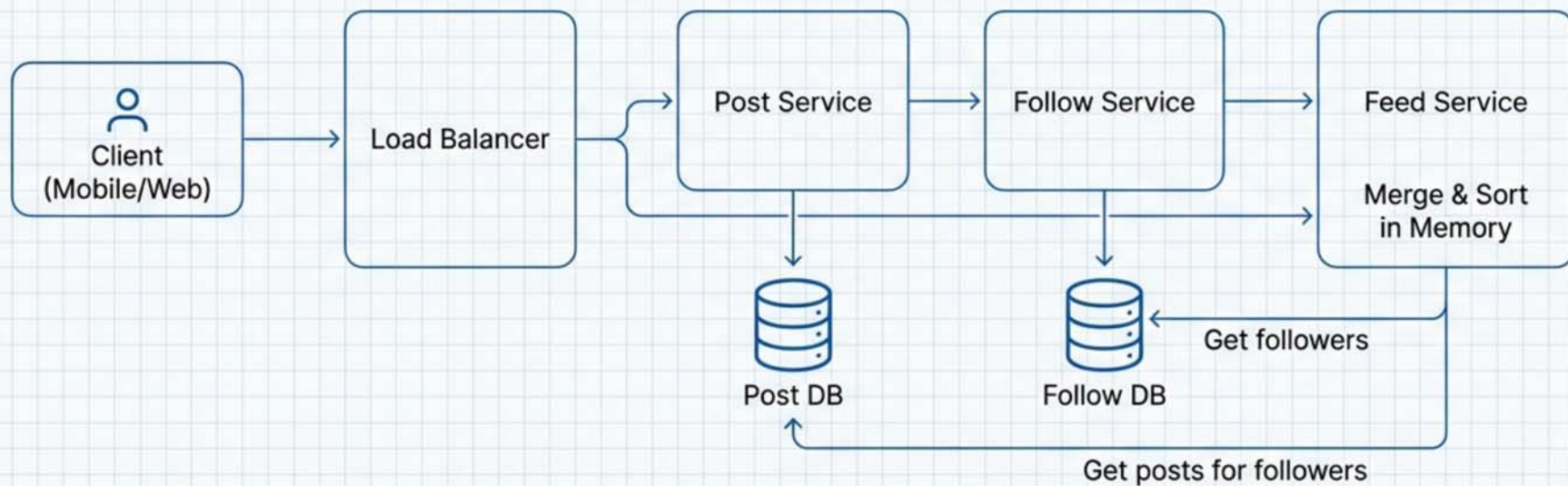
// 3. Get Feed
GET /feed?pageSize=25&cursor={timestamp}
Response: {
  items: Post[],
  nextCursor: string
}
```

Why Cursor-Based Pagination?

Offset pagination is unstable for real-time feeds where items are constantly inserted at the top. We use a 'timestamp' cursor to ensure the user doesn't see duplicates or miss posts as they scroll.

Step 4: High-Level Design (Naive Approach)

Fan-out on Read



Critique: This functional design places the heavy computational load on the "Read" path. The Feed Service must query and sort thousands of posts while the user waits.

Database Design (DynamoDB)

Modeling for Access Patterns

Table: Follows

Partition Key (PK): userFollowing

Sort Key (SK): userFollowed

GSI (Global Secondary Index): Reverse
(PK: userFollowed, SK: userFollowing)

Primary access: 'Who is User A following?' |
GSI access: 'Who follows User A?'

Table: Posts

Partition Key (PK): postId

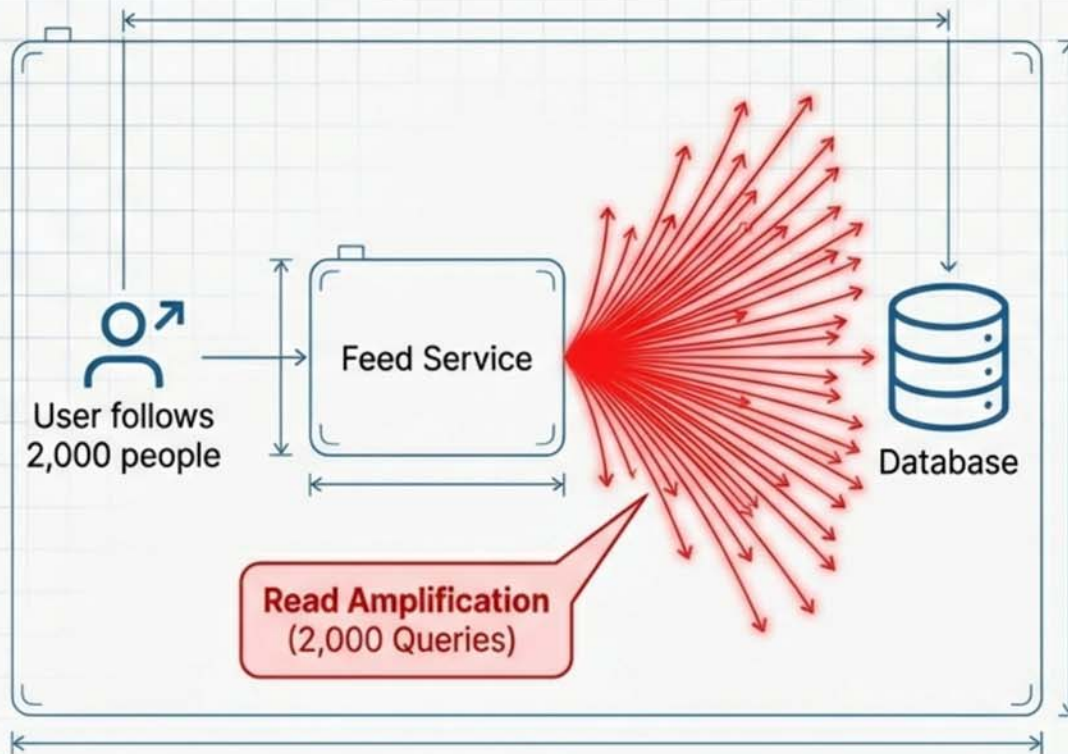
GSI Partition Key: creatorId

GSI Sort Key: creationTime

Why GSI? We need to efficiently fetch a
specific user's posts sorted by time.

Deep Dive 1: The Bottleneck

Why 'Fan-out on Read' Fails at Scale



The Scenario:

A user follows 2,000 accounts.

The Sequence:

1. Fetch 2,000 user IDs.
2. Execute 2,000 parallel queries against Post Table GSI.
3. Sort 10,000+ posts in memory.

The Result:

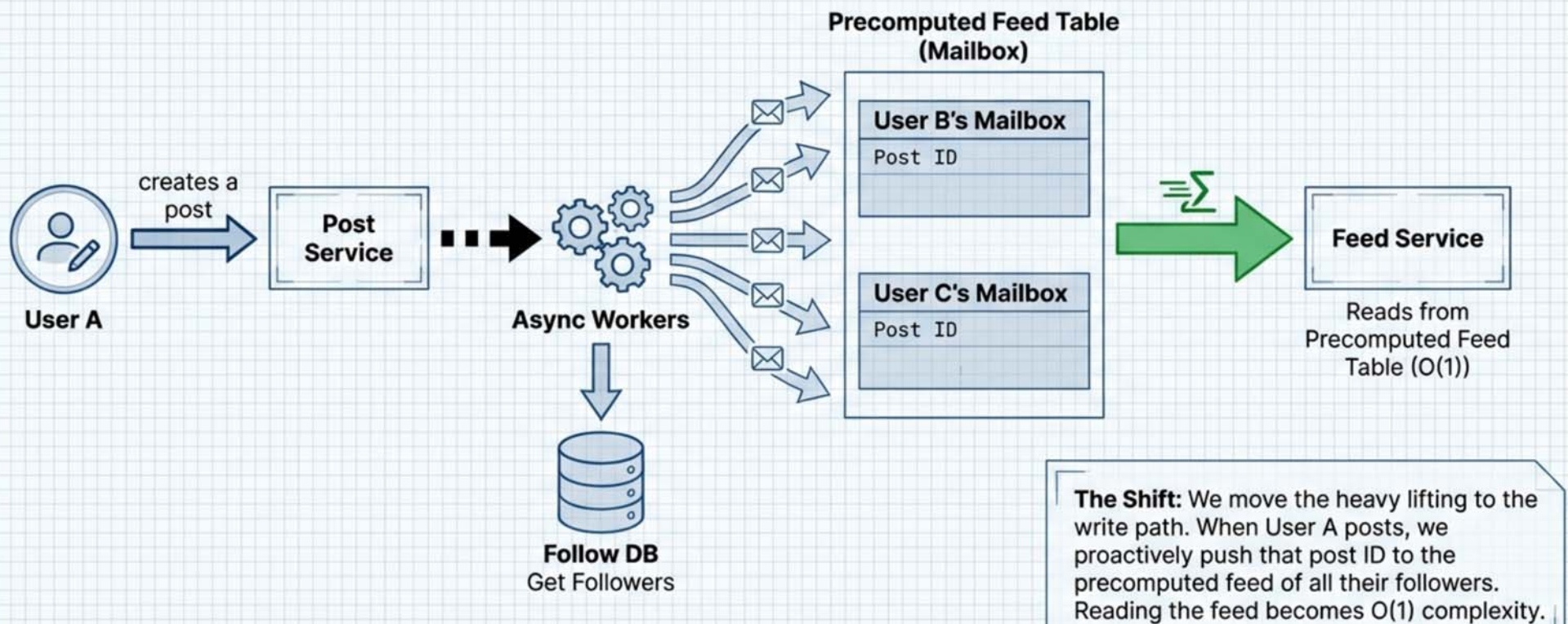
High latency, DB Throttling, Timeouts.

Verdict:

We are doing too much work on the "Hot Path" (while the user waits).

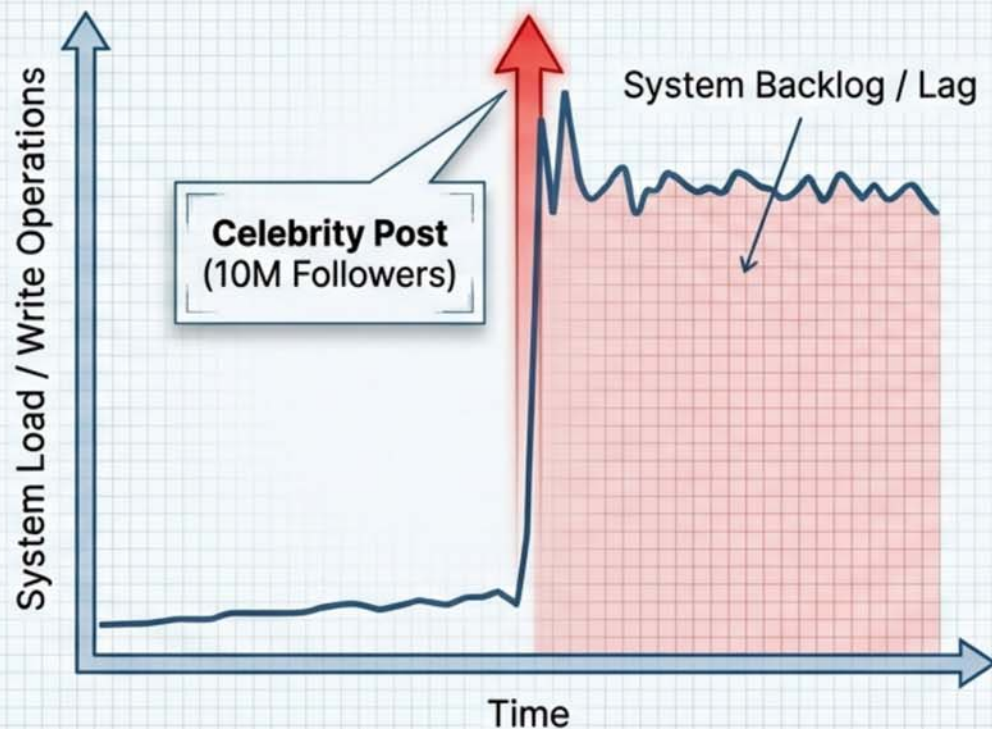
Solution: Fan-out on Write (Push Model)

Precomputing the Feed



Deep Dive 2: The 'Celebrity' Edge Case

When Push Models Break



The Problem: Write Amplification.

If Justin Bieber (10M followers) posts, the Async Workers try to generate 10,000,000 writes immediately.

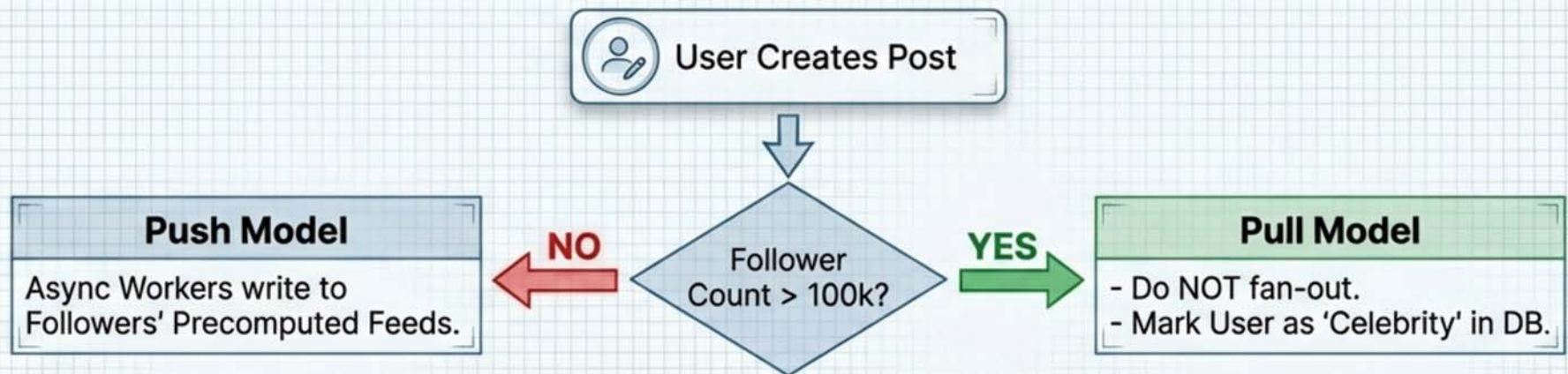
Consequences:

- Massive backlog in the worker queues.
- "The Thunderstorm of Activity".
- Violation of the 1-minute staleness requirement.

Conclusion: Fan-out on Write is great for 99% of users, but fatal for the top 1%.

The Ultimate Solution: Hybrid Approach

Combining Push and Pull



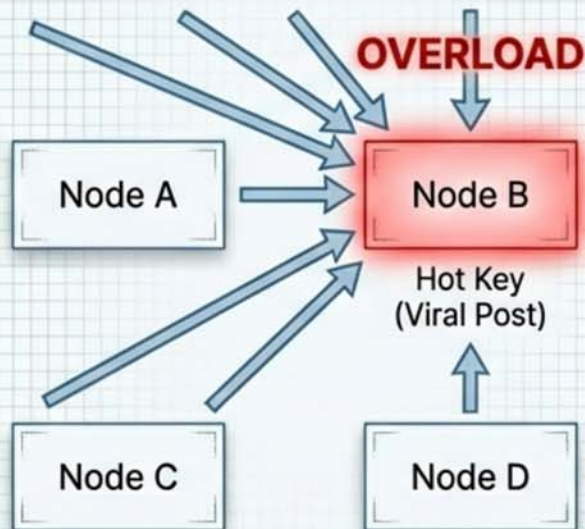
Feed Generation (Read Time)



Scaling Deep Dive: Hot Keys

Handling Viral Content

Standard Sharding (The Problem)

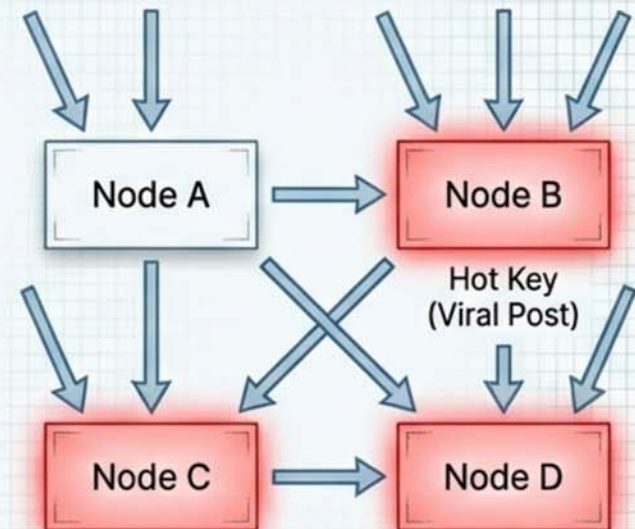


Viral Post ID maps to Single Shard → Overload.

Problem: A viral post is requested millions of times/sec. Even with caching, a single shard cannot handle the throughput.

Solution: Identify hot keys (High QPS) and replicate them across multiple cache instances. Randomly route requests to spread the load.

Replication (The Solution)



Distributed Load Balancing.

Final System Architecture

